EPSR-test-ex48 alignmole on a box, stdin input

Program(s): alignmole | chained passes: 1 | verification: ALL IDENTICAL (1 pass)

What this case tests

Exercises `alignmole` [1] on several molecules with longer runs: the `ran1` sequence [2,3] carried over molecules and runs, the step-size adaptation every `niter/10` iterations, axis atoms given on the command line or re-read from stdin, axis and iteration count from stdin, and one-atom molecules for which `rotatm` does nothing.

Code exercised

Fortran (original, fortran_src/):

- `alignmole.f`: `alignmole` (Monte Carlo loop, downhill-only acceptance, step-size adaptation every `niter/10` trials), `displacement` (mean square distance from the axis)
- `molfit.f`: `rotatm` (random rotation of a molecule about its centre); `auxiliary_routines.f`: `ran1`
- `ato_read_and_save.f`: `minitc`, `moloutc` (reading / rewriting the `.ato`, random-number state included)

Rust (port, rust/epsr-core/src/):

- `tools/alignmole.rs`: `run`, `displacement`
- `tools/groupc_common.rs`: `rotatm`, `moloutc`; `rng.rs`: `Ran1::ran1`
- `ato.rs`: `State::minitc`

Physical system

`box.ato`: 6 molecules in the order DMSO, water, DMSO, water, water, DMSO (39 atoms) in a cubic 25 Å box, each placed so that its first atom sits exactly at the molecule centre; with axis atoms `1 1` the displacement is identically zero, so every rotation is accepted while the molecules still rotate. `atoms.ato`: three one-atom molecules in a 20 Å box. Rotational step size 0.3 from the `box.ato` header.

Inputs

`box.ato` from a helper script (a one-off script that was not kept; the generated file is the input). Commands: `alignmole box.ato y 30 1 1` (step adapted at iterations 3, 6, ..., 30); `alignmole box.ato x 20 7 < box_atoms.stdin` (a single axis atom is not enough, the list `1 1` is read from stdin); `alignmole atoms.ato < atoms.stdin` (axis `z`, 20 iterations, all atoms).

How the accuracy was verified

`commands.txt` (3 program calls, run once) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in 1 pass.
Files compared: `box.ato` after both runs (coordinates and random-number state after 6 × 50 rotations) and `atoms.ato`. Compared against the original Fortran binary.

Notes

Limitation of the Fortran: a rejected trial rotation restores the molecule from the work arrays `xr, yr, zr`, which are allocated but never assigned, so uninitialised heap memory is copied into the coordinates; such runs cannot be reproduced (the port uses zero-filled arrays and prints a warning), and this case is designed so that no rotation is rejected. `niter` < `10` makes the Fortran divide by zero (`naccum` = `niter/10`); the port reports an input error.

References

[1] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[2] S. K. Park and K. W. Miller, "Random number generators: good ones are hard to find", Commun. ACM 31, 1192-1201 (1988).
[3] W. H. Press, S. A. Teukolsky, W. T. Vetterling and B. P. Flannery, Numerical Recipes in Fortran 77, 2nd ed., Cambridge University Press (1992).
[4] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.